1. **Задание**

   1 Переведите вебсервер на использование SSL

   2 Реализуйте form-based authentication

   3 Защитите ресурсы, требующие запись и введите аудит

2. **Описание продукта**

   Необходимо разработать форму логина для доступа в WEB-приложение и манипуляции данными хранилища литературы

3. **Листинг программного кода**

   GostApiController.java

```
@Slf4j
@Generated(value = "org.openapitools.codegen.languages.SpringCodegen", date = "2025-09-21T17:15:00.750982+03:00[Europe/Moscow]", comments = "Generator version: 7.7.0")
@Controller
@RequestMapping("${openapi..base-path:}")
@RequiredArgsConstructor
public class GostApiController implements GostApi {

    private final NativeWebRequest request;

    private final GostWorker GostWorker;

    @Override
    public Optional<NativeWebRequest> getRequest() {
        return Optional.ofNullable(request);
    }

    @Override
    public ResponseEntity<GOST> addBook(@Parameter(name = "GOST", description = "Добавление новой литературы", required = true) @Valid @RequestBody GOST GOSTBody, Principal principal)
    {
        if (principal == null) {
            throw new ResponseStatusException(HttpStatus.FORBIDDEN, "Доступ запрещён");
```

```
        }
        if (GOSTBody.getId() == null) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST).build();
        }

        if (GostWorker.IsGOSTExist(GOSTBody.getId())) {
            return ResponseEntity.status(HttpStatus.CONFLICT).build();
        }

        log.info("User {} добавил литературу: {}", principal.getName(), GOSTBody);

        GostWorker.AddGOST(GOSTBody.getId(), GOSTBody);
        return ResponseEntity.ok(GOSTBody);
    }

    @Override
    public ResponseEntity<Void> deleteBook(@NotNull @Parameter(name = "id",
description = "идентификатор литературы", required = true, in = ParameterIn.QUERY)
@Valid @RequestParam(value = "id", required = true) Long id, Principal principal)
    {
        if (principal == null) {
            throw new ResponseStatusException(HttpStatus.FORBIDDEN, "Доступ
запрещён");
        }
        if (id == null) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST).build();
        }
        if (!GostWorker.IsGOSTExist(id)) {
            return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
        }

        log.info("User {} удалил литературу по ID {}", principal.getName(), id);
        GostWorker.DeleteById(id);
        return null;
    }
```

```
    @Override
    public ResponseEntity<List<GOST>> getBooks(Principal principal)
    {
        if (principal != null) {
            log.info("User {} запросил весь список литературы", principal.getName());
        }
        return ResponseEntity.ok(GostWorker.ShowAll());
    }
}
```

ControllersConfig.java

```
@Configuration
public class ControllersConfig implements WebMvcConfigurer {

    @Override
    public void addViewControllers(ViewControllerRegistry registry) {
        registry.addViewController("/login").setViewName("login");
        registry.addViewController("/gostPage").setViewName("gost");
    }

}
```

PasswordEncoderConfig.java

```
@Configuration
public class PasswordEncoderConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

SecurityConfiguration.java

```
@Configuration
@EnableWebSecurity
public class SecurityConfiguration {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests(authorizationManagerRequestMatcherRegistry ->
authorizationManagerRequestMatcherRegistry.requestMatchers("/js/**", "/css/**",
"/gostPage", "/gost")
                        .permitAll()
                        .anyRequest()
                        .authenticated())
                .formLogin(httpSecurityFormLoginConfigurer ->
httpSecurityFormLoginConfigurer
                        .loginPage("/login")
                        .usernameParameter("login")
                        .passwordParameter("pass")
                        .permitAll()
                        .defaultSuccessUrl("/gostPage", true))
                .logout(httpSecurityLogoutConfigurer ->
httpSecurityLogoutConfigurer.permitAll()
                        .logoutRequestMatcher(new AntPathRequestMatcher("/logout"))
                        .logoutSuccessUrl("/login"));
        return http.build();
    }

    @Autowired
    public void configureGlobal(AuthenticationManagerBuilder auth, PasswordEncoder
passwordEncoder) throws Exception {
        auth.inMemoryAuthentication()
                .withUser("admin")
                .password(passwordEncoder.encode("admin"))
                .roles("ALL");
    }
```

```
}
```

Gost.js

```
var app = angular.module("gost", []);

app.run(function($http) {
    var csrftoken = document.querySelector("meta[name='_csrf']").getAttribute("content");
    var csrfheader =
document.querySelector("meta[name='_csrf_header']").getAttribute("content");
    $http.defaults.headers.common["X-CSRF-TOKEN"] = csrftoken;
});

app.controller("GostController", function ($scope, $http) {

    $scope.successGetGost = function (response) {
        $scope.GostList = response.data;
        $scope.errormessage="";
    };

    $scope.errorGetGost = function (error) {
        console.log(error);
        $scope.errormessage="ERROR. Не удалось получить список";
    };

    $scope.getGosts = function () {
        $http.get('/gost').then($scope.successGetGost, $scope.errorGetGost);
    };

    $scope.successDeleteGostCallback = function (response) {
        for (var i = 0; i < $scope.GostList.length; i++) {
            var j = $scope.GostList[i];
            if (j.id === $scope.deletedId) {
                $scope.GostList.splice(i, 1);
                break;
```

```
            }
        }
        $scope.errormessage="";
    };

    $scope.errorDeleteGostCallback = function (error) {
        console.log(error);
        if (error.status === 405) {
            $scope.errormessage = "У вас нет прав";
        } else
        if (error.status === 403) {
            $scope.errormessage = "У вас нет прав";
        } else
        $scope.errormessage="ERROR. Не удалось удалить этот ID";
    };

    $scope.deleteGost = function (id) {
        $scope.deletedId = id;
        $http.delete('/gost?id=' + id).then($scope.successDeleteGostCallback,
$scope.errorDeleteGostCallback);
    };

    $scope.successAddNewGostCallback = function (response) {
        $scope.getGosts();
        $scope.errormessage="";
    };

    $scope.errorAddNewGostCallback = function (error) {
        console.log(error);
        if (error.status === 405) {
            $scope.errormessage = "У вас нет прав";
        } else
        if (error.status === 403) {
            $scope.errormessage = "У вас нет прав";
        } else
```

```
        $scope.errormessage="ERROR. Не удалось добавить эту информацию";
    };

    $scope.addNewGost = function () {
        var data = {
            "id": $scope.id,
            "year": $scope.year,
            "author": $scope.author,
            "name": $scope.name
        };
        console.log(data);
        $http.post('/gost', data)
            .then($scope.successAddNewGostCallback, $scope.errorAddNewGostCallback);
    };

});
```

4. **Выводы**

В процессе лабораторной работы были получены практические навыки использования протоколов безопасности средствами Spring Boot